

Lectura 3.1: Modos de falla y estrategias de mejora para PINN

Introducción a las redes neuronales informadas por física
Especialización en Inteligencia Artificial - B32025

Outline

- Generalidades
- Adimensionalización
- Implementación “soft” y “hard” de CB Dirichlet y de CI
- Estrategia de entrenamiento auto-adaptativa
- Estrategias de muestreo basadas en residuos

Dificultades en el entrenamiento de PINN (“modos de falla”)

Artículos destacados:

- Wang et al. An expert's guide to training PINN.
- Krishnapriyan et al. Characterizing possible failure modes in PINN.
- Wang et al. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks.
- Monaco & Apletti. Training physics-informed neural networks: One learning to rule them all?

Dificultades en el entrenamiento de PINN (“modos de falla”)

Algunas causas principales:

- Desbalance en los distintos términos de la función de pérdida
- Desbalance en las magnitudes de los gradientes de los dichos términos
- Conflicto en las direcciones de los gradientes de dichos términos
- Espectral bias inherente a las NN

Estos modos de falla no están asociados con una “falta de expresividad” de las redes neuronales

Problema típico: tamaño relativo de los sub-conjuntos de entrenamiento

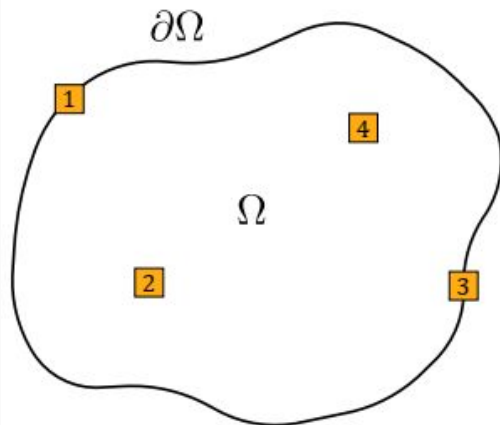
$$\begin{aligned}
 X_{\text{PDE}} &= \{P_i = (\mathbf{x}_i, t_i) / P_i \in \Omega \times (t_0, t_f]; i = 1, \dots, N_{\text{PDE}}\}, \\
 X_{\text{BC}} &= \{Q_i = (\mathbf{x}_i, t_i) / Q_i \in \partial\Omega \times (t_0, t_f]; i = 1, \dots, N_{\text{BC}}\}, \\
 X_{\text{IC}} &= \{R_i = (\mathbf{x}_i, t_0) / R_i \in \Omega \cup \partial\Omega; i = 1, \dots, N_{\text{IC}}\}, \\
 X_{\text{data}} &= \{S_i = (\mathbf{x}_i, t_i) / S_i \in \Omega \cup \partial\Omega \times [t_0, t_f]; i = 1, \dots, N_{\text{data}}\}.
 \end{aligned}$$

$$X^i = \{X_{\text{PDE}}^i, X_{\text{BC}}^i, X_{\text{IC}}^i, X_{\text{data}}^i\},$$

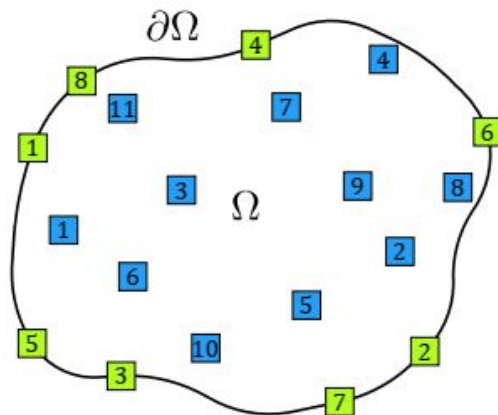
$$N_{\text{PDE}} = 11, N_{\text{BC}} = 8, N_{\text{IC}} = 4 \text{ y } N_{\text{data}} = 7.$$

$$N^i = N_{\text{PDE}}^i + N_{\text{BC}}^i + N_{\text{IC}}^i + N_{\text{data}}^i,$$

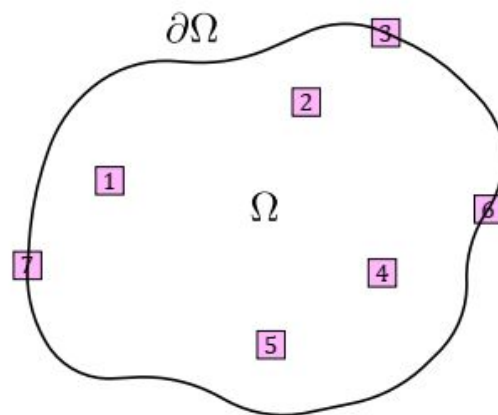
$$N_{\text{PDE}}^i \neq N_{\text{BC}}^i \neq N_{\text{IC}}^i \neq N_{\text{data}}^i.$$



(A) $X_{\text{IC}} (t = t_0)$.

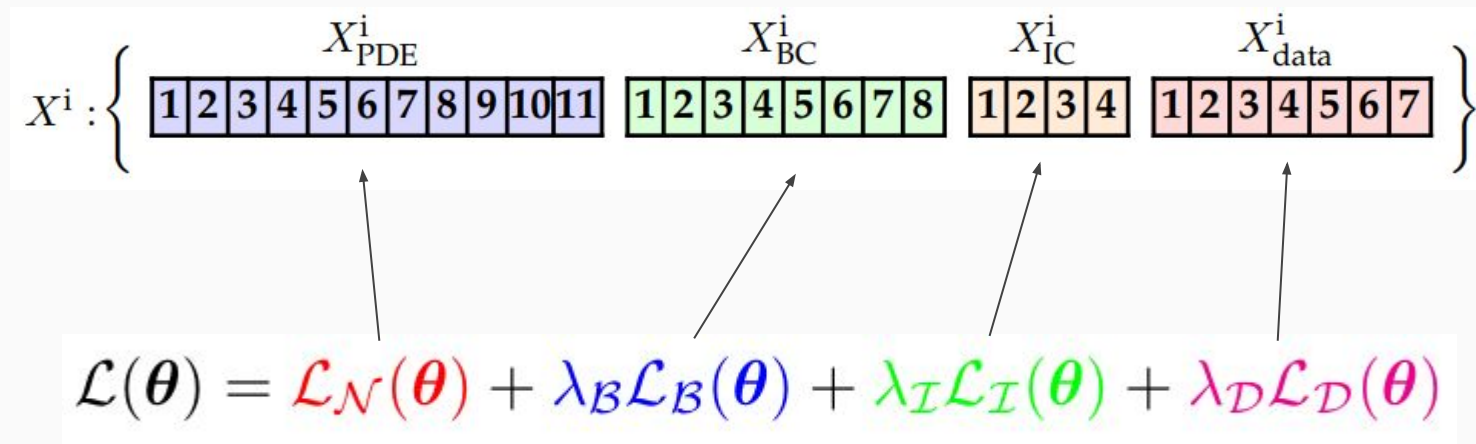


(B) $X_{\text{PDE}} \cup X_{\text{BC}} (t \in (t_0, t_f])$.



(C) $X_{\text{data}} (t \in [t_0, t_f])$.

Problema típico: tamaño relativo de los sub-conjuntos de entrenamiento



¿Cómo elegir los pesos λ ? ¿Se pueden elegir de una forma “inteligente”?
¿Es posible eliminar estas restricciones impuestas de manera “soft”? Todo esto y más en la clase de hoy...

Problema típico: tamaño relativo de los sub-conjuntos de entrenamiento

Además, aparecen también algunos aspectos en la implementación...

¿Puedo entrenar usando un dataset concatenado?

$$X^i : \left\{ \begin{array}{cccc} X_{PDE}^i & X_{BC}^i & X_{IC}^i & X_{data}^i \end{array} \right\}$$


¿Cómo procedería en caso de querer entrenar con mini-batches?

$$\begin{aligned} X_1^i &: \left\{ \begin{array}{|c|c|c|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ \hline \end{array} \right\} \\ X_2^i &: \left\{ \begin{array}{|c|c|c|} \hline 9 & 10 & 11 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 3 & 4 & 5 \\ \hline \end{array} \right\} \\ X_3^i &: \left\{ \begin{array}{|c|c|c|} \hline 6 & 7 & 8 \\ \hline \end{array} \quad \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 1 \\ \hline \end{array} \right\} \\ X_4^i &: \left\{ \begin{array}{|c|c|c|c|c|c|} \hline 2 & 3 & 4 & 5 & 6 & 7 \\ \hline \end{array} \right\} \end{aligned}$$

(A) Minibatches X_j^i conformados a partir de la técnica estándar.

$$\begin{aligned} X_1^i &: \left\{ \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 1 & 2 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 1 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 1 & 2 \\ \hline \end{array} \right\} \\ X_2^i &: \left\{ \begin{array}{|c|c|c|} \hline 4 & 5 & 6 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 3 & 4 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 2 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 3 & 4 \\ \hline \end{array} \right\} \\ X_3^i &: \left\{ \begin{array}{|c|c|c|} \hline 7 & 8 & 9 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 5 & 6 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 3 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 5 & 6 \\ \hline \end{array} \right\} \\ X_4^i &: \left\{ \begin{array}{|c|c|} \hline 10 & 11 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline 7 & 8 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 4 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 7 \\ \hline \end{array} \right\} \end{aligned}$$

(B) minibatches X_j^i conformados a partir de la técnica propuesta.

Problema típico: tamaño relativo de los sub-conjuntos de entrenamiento

- En casos “de juguete” podemos emplear la cantidad de puntos de colocación que querramos
- En problemas “reales”, esto no es cierto (más si vamos a emplear datos rotulados)
- Se deben emplear estrategias específicas para tratar problemas de desbalance

Estrategias “paliativas”

Adimensionalización

- Remoción de unidades físicas mediante una sustitución de variables adecuada
- Estrechamente relacionado al análisis dimensional (Teorema “Pi”)
- Números adimensionales: Fo , Re , Pe , Nu , etc
- Mitiga la disparidad entre las distintas escalas → Mejora la convergencia, aunque no es “bala de plata”
- **Adimensionalización $\neq$ Normalización**

Números adimensionales (fuente: Wikipedia)

Name	Standard symbol	Definition	Field of application
Biot number	Bi	$Bi = \frac{hL_C}{k_b}$	heat transfer (surface vs. volume conductivity of solids)
Drag coefficient	c_d	$c_d = \frac{2F_d}{\rho v^2 A}$,	aeronautics, fluid dynamics (resistance to fluid motion)
Nusselt number	Nu	$Nu = \frac{hd}{k}$	heat transfer (forced convection; ratio of convective to conductive heat transfer)
Péclet number	Pe	$Pe = \frac{Lu}{D}$ or $Pe = \frac{Lu}{\alpha}$	fluid mechanics (ratio of advective transport rate over molecular diffusive transport rate), heat transfer (ratio of advective transport rate over thermal diffusive transport rate)
Prandtl number	Pr	$Pr = \frac{\nu}{\alpha} = \frac{c_p \mu}{k}$	heat transfer (ratio of viscous diffusion rate over thermal diffusion rate)
Pressure coefficient	C_p	$C_p = \frac{p - p_\infty}{\frac{1}{2} \rho_\infty V_\infty^2}$	aerodynamics, hydrodynamics (pressure experienced at a point on an airfoil; dimensionless pressure variable)
Rayleigh number	Ra	$Ra_x = \frac{g\beta}{\nu\alpha} (T_s - T_\infty) x^3$	heat transfer (buoyancy versus viscous forces in free convection)
Reynolds number	Re	$Re = \frac{UL\rho}{\mu} = \frac{UL}{\nu}$	fluid mechanics (ratio of fluid inertial and viscous forces) ^[5]

Adimensionalización - Ejemplo

Ec. del calor transiente no homogénea con coef. constantes

$$\frac{\partial T}{\partial t} - \alpha \frac{\partial^2 T}{\partial x^2} = q$$

Cambio de variables en función magnitudes características

$$\xi = \frac{x}{L}, \quad \tau = Fo \equiv \frac{\alpha t}{L^2}, \quad u = \frac{T}{T_0}$$

Reemplazo

$$\frac{T_0 \partial u}{(L^2/\alpha) \partial \tau} - \alpha \frac{T_0 \partial^2 u}{L^2 \partial \xi^2} = q$$

Ec. del calor adimensional resultante

$$\frac{\partial u}{\partial \tau} - \frac{\partial^2 u}{\partial \xi^2} = \frac{L^2}{\alpha T_0} q \equiv \varphi$$

Implementación “soft” de CB Dirichlet y de CI

Hasta el momento hemos visto la implementación “soft” dada por:

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{\text{PDE}}(\boldsymbol{\theta}) + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}(\boldsymbol{\theta}) + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}(\boldsymbol{\theta})$$

- No se asegura que las restricciones se satisfagan idénticamente
- Esto puede ser un problema en problemas mal colocados (muy sensibles frente a cambios en CB o CI)
- Hay alguna manera de mejorar esto?

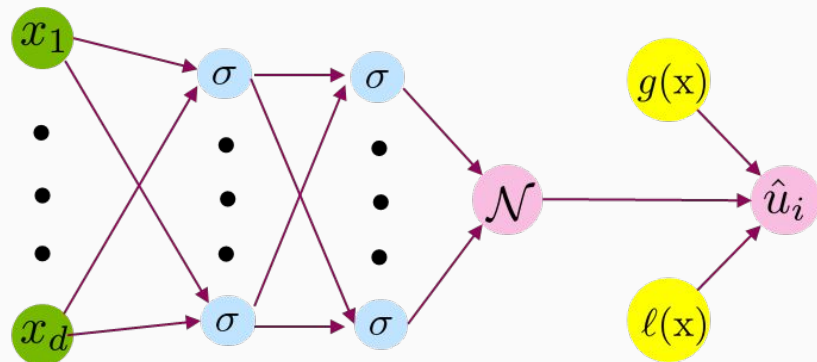
Implementación “hard” de CB Dirichlet y de CI

Consideremos una BC o IC dada por: $u_i(\mathbf{x}) = g_0(\mathbf{x}), \quad \mathbf{x} \in \Gamma_D,$

Solución de “trial”: $\hat{u}_i(\mathbf{x}; \boldsymbol{\theta}_u) = g(\mathbf{x}) + \ell(\mathbf{x})\mathcal{N}(\mathbf{x}; \boldsymbol{\theta}_u),$

$g(\mathbf{x})$ es una extensión continua de $g_0(\mathbf{x})$ de Γ_D hacia Ω , mientras que $\ell(\mathbf{x})$ es una función que satisface las siguientes condiciones:

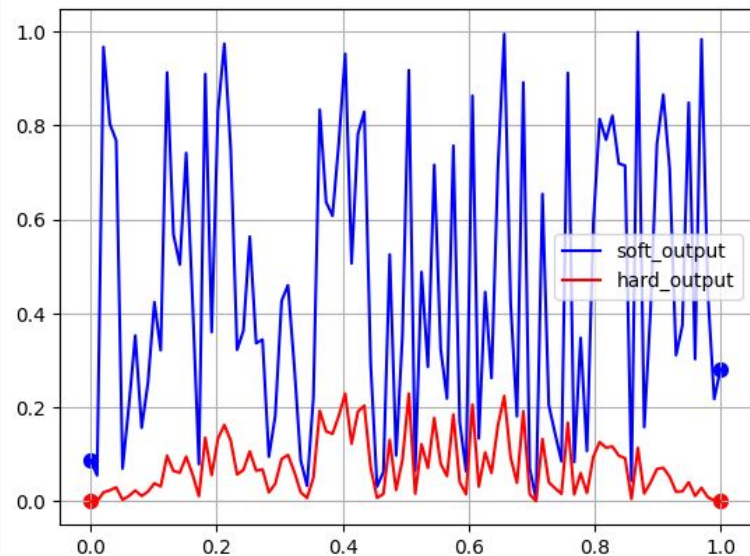
$$\begin{cases} \ell(\mathbf{x}) = 0, & \mathbf{x} \in \Gamma_D \\ \ell(\mathbf{x}) > 0, & \mathbf{x} \in \Omega - \Gamma_D \end{cases}$$



Ejemplo implementación “hard” de CB Dirichlet y de CI

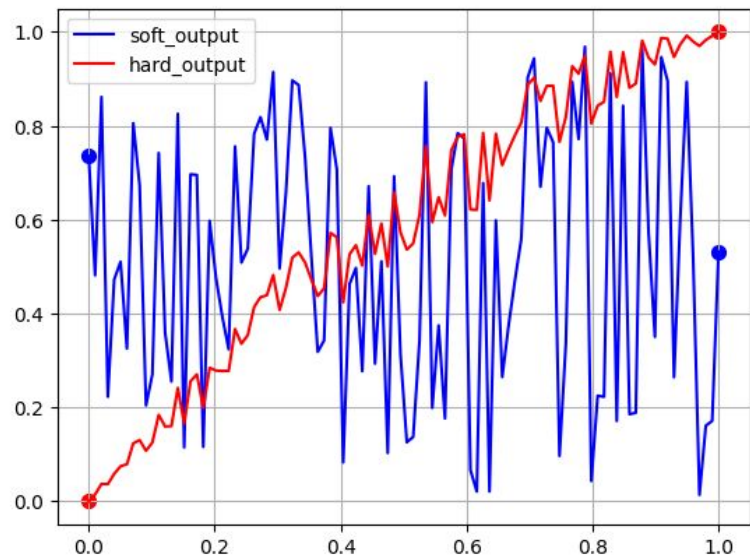
$$u(0) = u(1) = 0 : \quad g(x) = 0, l(x) = x(1 - x)$$

```
4 input = np.linspace(0, 1, 100).reshape(-1, 1)
5 output = np.random.rand(len(input), 1)
6 hard_bc = lambda x, out: x*(1-x)*out
7 output_hard_bc = hard_bc(input, output)
```



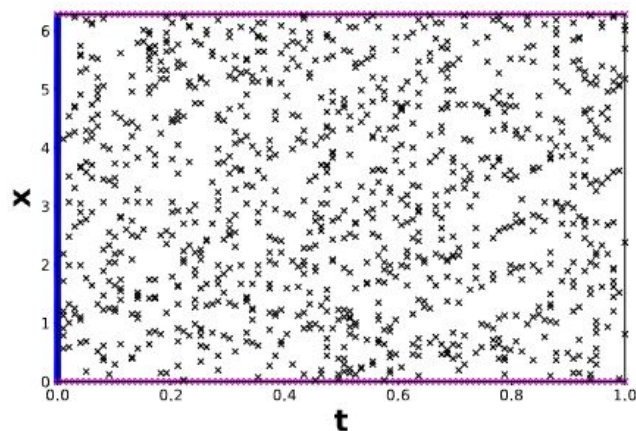
$$u(0) = 0, u(1) = 1 : g(x) = x, l(x) = x(1 - x)$$

```
4 input = np.linspace(0, 1, 100).reshape(-1, 1)
5 output = np.random.rand(len(input), 1)
6 hard_bc = lambda x, out: x + x*(1-x)*out
7 output_hard_bc = hard_bc(input, output)
```

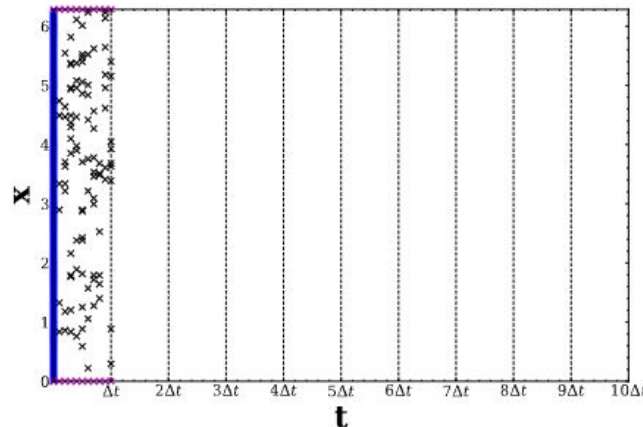


Respetar causalidad temporal

- La solución de PDEs transientes depende fuertemente de la causalidad temporal
- Vanilla PINN se entrena utilizando los instantes de tiempo a la vez, rompiendo la dependencia temporal $\rightarrow$ solución sesgada hacia predicciones a $t > 0$
- Alternativa: entrenamiento *seq2seq*:



(a) *Regular PINN training*



(b) *Sequence-to-sequence learning (model trained every Δt)*

× Initial condition points × Boundary points × Collocation points

Implica entrenar “N”
redes neuronales
(una para cada
porción del dominio
temporal)

Respetar causalidad temporal

- Resulta necesario imponer la estructura causal faltante
- Alternativa (computacionalmente eficiente): consiste en subdividir el dominio temporal en “M” segmentos secuenciales (idem seq2seq), y computar en cada uno de ellos la función de pérdida $\mathcal{L}_r^i$.
- Luego, escribir la función de pérdida para todo el dominio temporal como:

$$\mathcal{L}_r(\theta) = \frac{1}{M} \sum_{i=1}^M w_i \mathcal{L}_r^i(\theta) = \frac{1}{M} \sum_{i=1}^M \exp \left(-\epsilon \sum_{k=1}^{i-1} \mathcal{L}_r^k(\theta) \right) \mathcal{L}_r^i(\theta).$$

- w_i es inversa y exponencialmente proporcional a la magnitud de la pérdida residual acumulada de pasos de tiempo previos
- $\mathcal{L}_r^i$ no puede ser minimizada a menos que todos los residuos previos decrezcan a un valor suficientemente pequeño tal que w_i sean grandes

Respetar causalidad temporal

$$\mathcal{L}_r(\theta) = \frac{1}{M} \sum_{i=1}^M w_i \mathcal{L}_r^i(\theta) = \frac{1}{M} \sum_{i=1}^M \exp \left(-\epsilon \sum_{k=1}^{i-1} \mathcal{L}_r^k(\theta) \right) \mathcal{L}_r^i(\theta).$$

- La estrategia propuesta es sensible a la elección del parámetro ϵ
- $\epsilon \rightarrow 0$ implica recuperar comportamiento “vanilla PINN”
- $\epsilon \rightarrow \infty$ hace muy difícil el entrenamiento
- Otras estrategias: Causality-enhanced Discrete PINN, etc...

Ejemplo - Solución de la ecuación de Allen-Cahn mediante PINN

$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad t \in [0, 1], x \in [-1, 1],$$

$$u(x, 0) = x^2 \cos(\pi x),$$

$$u(t, -1) = u(t, 1),$$

$$u_x(t, -1) = u_x(t, 1).$$

Datos del modelo:

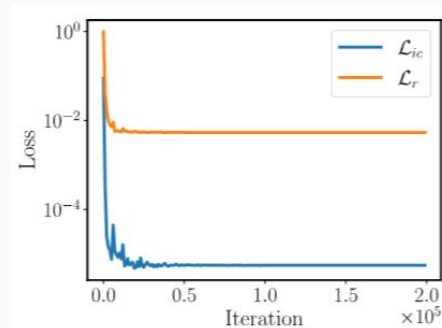
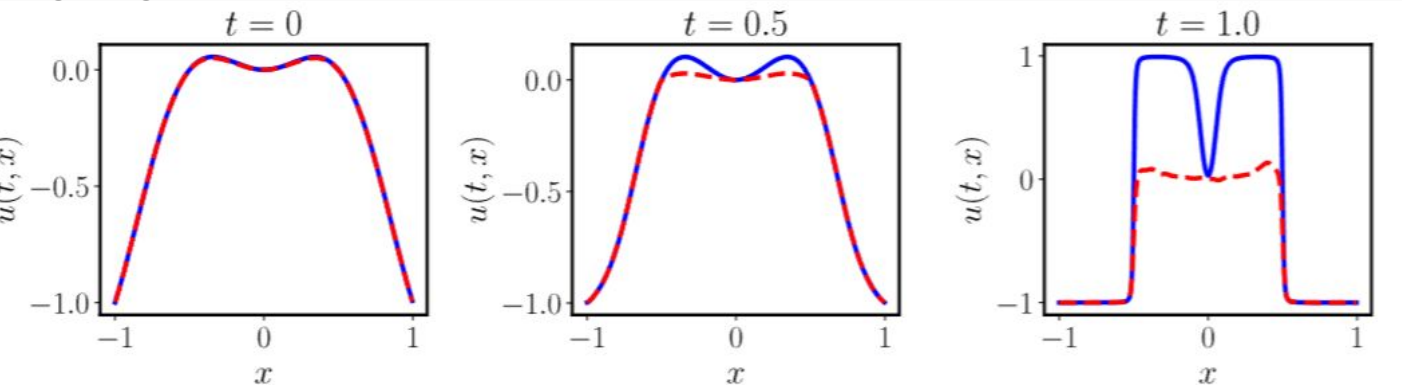
- N_dom: 25600 puntos
- N_ic: 256 puntos
- \lambda_{ic}=100, \lambda_r=1
- Optimizador Adam, 2e5 iteraciones
- Arquitectura FC-FFNN: 4 hidden layers, 128 neuronas por capa. Función de activación tanh()

$$\mathcal{L}(\theta) = \lambda_{ic}\mathcal{L}_{ic}(\theta) + \lambda_r\mathcal{L}_r(\theta),$$

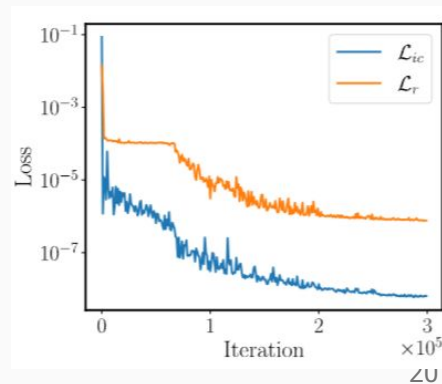
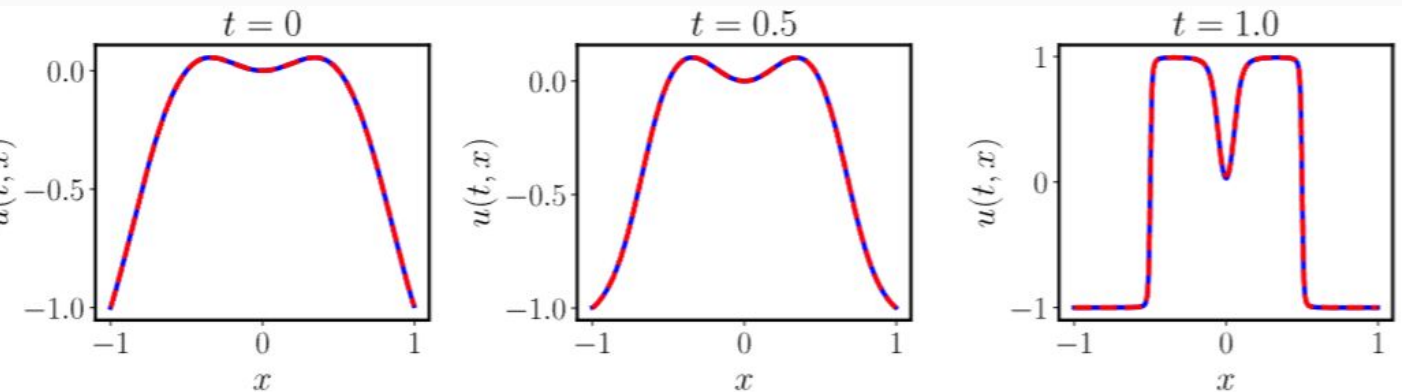
(CB periódicas implementadas de manera "hard")

Ejemplo - Solución de la ecuación de Allen-Cahn mediante PINN

Vanilla PINN



PINN que respeta causalidad



Uso de arquitecturas recurrentes (RNN, LSTM, GRU)

- Entrenamiento seq2seq: partición del dominio temporal en “N” subconjuntos $t_k, k = 1, \dots, N$
- La red predice la respuesta en cada secuencia, y esta es alimentada a la secuencia siguiente
- LSTM: implica definir:
 - h_k : estado “oculto” en t_k
 - c_k : estado “de celda” en t_k
 - En cada t_k : $h_k, c_k = \text{LSTM}(x, h_{k-1}, c_{k-1}; \theta)$
 - Capa de salida mapea el estado oculto hacia la predicción: $u_k = \phi(h_k; \theta_u)$
 - El modelo en cada t_k es: $\hat{u}(x, t_k) = u_k$

Pérdida para la IC:

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{IC}}} \sum_{i=1}^{N_{\text{IC}}} |\hat{u}(x_i, t_0) - u_0(x_i)|^2$$

Pérdida para la BC:

$$\mathcal{L}_{\text{BC}}^{(k)} = \frac{1}{N_{\text{BC}}} \sum_{j=1}^{N_{\text{BC}}} |\hat{u}(x_j^{\text{BC}}, t_k) - g(x_j^{\text{BC}}, t_k)|^2$$

Pérdida para la PDE:

$$\mathcal{L}_{\text{PDE}}^{(k)} = \frac{1}{N_f} \sum_{l=1}^{N_f} |\mathcal{N}[\hat{u}(x_l, t_k)]|^2$$

$$\mathcal{L} = \mathcal{L}_{\text{IC}} + \sum_{k=1}^N \left(\mathcal{L}_{\text{BC}}^{(k)} + \mathcal{L}_{\text{PDE}}^{(k)} \right)$$

Arquitectura PI-LSTM

Ventajas:

- Maneja naturalmente el aprendizaje de secuencias temporales.
- Comparte parámetros a lo largo del tiempo — lo que lo hace eficiente y compacto.
- Aprende el comportamiento latente del avance temporal (time-stepping).
- Puede generalizar mejor a horizontes temporales largos que los PINNs con redes totalmente conectadas (en ciertos casos).

Desventajas:

- El entrenamiento es más complejo debido a la dependencia secuencial.
- Es necesario utilizar Backpropagation Through Time (BPTT).
- Sensible a la inicialización y a la elección del optimizador.
- Puede acumular errores numéricos a lo largo del tiempo, a menos que el residuo de la EDP se imponga fuertemente

Balanceo de términos de pérdida

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}}(\theta) + \lambda_{\text{BC}}\mathcal{L}_{\text{BC}}(\theta) + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}(\theta)$$

- Balanceo manual de los pesos λ es impráctico y proclive a errores
- Empíricamente, suele emplearse λ_{IC} y $\lambda_{\text{BC}} \gg 1$ (10 o más)
- Estrategia planteada por Wang et al.:

$$\hat{\lambda}_{ic} = \frac{\|\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_{bc}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_r(\theta)\|}{\|\nabla_{\theta}\mathcal{L}_{ic}(\theta)\|},$$

$$\hat{\lambda}_{bc} = \frac{\|\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_{bc}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_r(\theta)\|}{\|\nabla_{\theta}\mathcal{L}_{bc}(\theta)\|},$$

$$\hat{\lambda}_r = \frac{\|\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_{bc}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_r(\theta)\|}{\|\nabla_{\theta}\mathcal{L}_r(\theta)\|},$$

$$\lambda_{\text{new}} = \alpha\lambda_{\text{old}} + (1 - \alpha)\hat{\lambda}_{\text{new}}$$

Actualización de los pesos puede hacerse cada “n” epochs

$$\|\hat{\lambda}_{ic}\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| = \|\hat{\lambda}_{bc}\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| = \|\hat{\lambda}_r\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| = \|\nabla_{\theta}\mathcal{L}_{ic}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_{bc}(\theta)\| + \|\nabla_{\theta}\mathcal{L}_r(\theta)\|$$

Curriculum training

Propuesto por Krishnapriyan et al. (Characterizing possible failure modes...), basado en el método análogo a “curriculum learning” en ML

Ejemplo - Aprendizaje del problema de convección con CB periódicas, siendo β el coeficiente de convección:

$$\begin{aligned} \frac{\partial u}{\partial t} + \beta \frac{\partial u}{\partial x} &= 0, \quad x \in \Omega, \quad t \in [0, T], \\ u(x, 0) &= h(x), \quad x \in \Omega. \end{aligned}$$

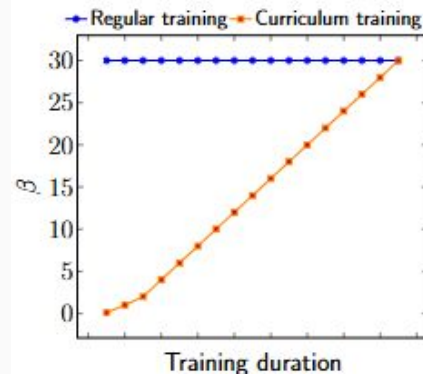
$$\begin{aligned} u(x, 0) &= \sin(x), \\ u(0, t) &= u(2\pi, t). \end{aligned}$$

$$u_{\text{analytical}}(x, t) = F^{-1}(F(h(x))e^{-i\beta kt}),$$

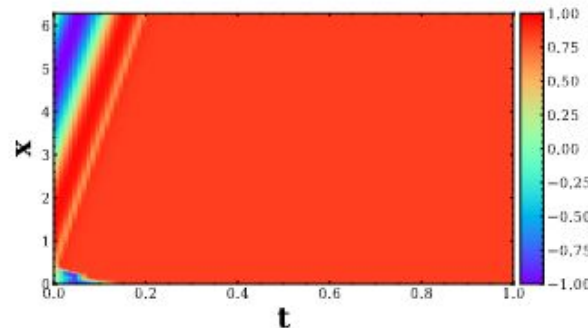
$$\mathcal{L}(\theta) = \frac{1}{N_u} \sum_{i=1}^{N_u} \left(\hat{u} - u_0^i \right)^2 + \frac{1}{N_f} \sum_{i=1}^{N_f} \lambda_i \left(\frac{\partial \hat{u}}{\partial t} + \beta \frac{\partial \hat{u}}{\partial x} \right)^2 + \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\hat{u}(\theta, 0, t) - \hat{u}(\theta, 2\pi, t) \right)^2.$$

Curriculum training

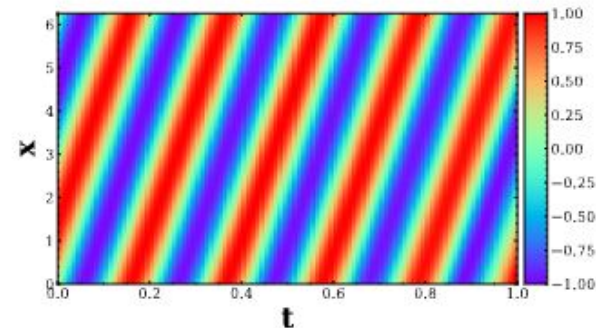
β	1	10	20	30	40
Relative error	7.84×10^{-3}	1.08×10^{-2}	7.50×10^{-1}	8.97×10^{-1}	9.61×10^{-1}
Absolute error	3.17×10^{-3}	6.03×10^{-3}	4.32×10^{-1}	5.42×10^{-1}	5.82×10^{-1}



(a) Curriculum regularization schematic



(b) Regular training PINN solution for $\beta = 30$



(c) Curriculum training PINN solution for $\beta = 30$

Estrategia de entrenamiento auto-adaptativa (SA-PINN)

Propuesta por: McClenny L, Braga-Neto U. Self-adaptive physics-informed neural networks using a soft attention mechanism. arXiv preprint arXiv:2009.04544. 2020.

PVIB a resolver:

$$\begin{aligned}\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x},t)] &= f(\mathbf{x},t), \quad \mathbf{x} \in \Omega, \quad t \in (0,T], \\ \mathcal{B}_{\mathbf{x},t}[u(\mathbf{x},t)] &= g(\mathbf{x},t), \quad \mathbf{x} \in \partial\Omega, \quad t \in (0,T], \\ u(\mathbf{x},0) &= h(\mathbf{x}), \quad \mathbf{x} \in \bar{\Omega}.\end{aligned}$$

Función de pérdida global de PINN (vanilla):

$$\mathcal{L}(\mathbf{w}) = \mathcal{L}_s(\mathbf{w}) + \mathcal{L}_r(\mathbf{w}) + \mathcal{L}_b(\mathbf{w}) + \mathcal{L}_0(\mathbf{w})$$

Funciones de pérdida:

$$\begin{aligned}\mathcal{L}_s(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^{N_s} |u(\mathbf{x}_s^i, t_s^i; \mathbf{w}) - y_s^i|^2, \\ \mathcal{L}_r(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^{N_r} |\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x}_r^i, t_r^i; \mathbf{w})] - f(\mathbf{x}_r^i, t_r^i)|^2, \\ \mathcal{L}_b(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^{N_b} |\mathcal{B}_{\mathbf{x},t}[u(\mathbf{x}_b^i, t_b^i; \mathbf{w})] - g(\mathbf{x}_b^i, t_b^i)|^2, \\ \mathcal{L}_0(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^{N_0} |u(\mathbf{x}_0^i, 0; \mathbf{w}) - h(\mathbf{x}_0^i)|^2.\end{aligned}$$

Estrategia de entrenamiento auto-adaptativa (SA-PINN)

Los autores proponen la siguiente función de pérdida modificada:

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\lambda}_r, \boldsymbol{\lambda}_b, \boldsymbol{\lambda}_0) = \mathcal{L}_s(\mathbf{w}) + \mathcal{L}_r(\mathbf{w}, \boldsymbol{\lambda}_r) + \mathcal{L}_b(\mathbf{w}, \boldsymbol{\lambda}_b) + \mathcal{L}_0(\mathbf{w}, \boldsymbol{\lambda}_0)$$

$$\begin{aligned}\mathcal{L}_r(\mathbf{w}, \boldsymbol{\lambda}_r) &= \frac{1}{2} \sum_{i=1}^{N_r} m(\lambda_r^i) |\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x}_r^i, t_r^i; \mathbf{w})] - f(\mathbf{x}_r^i, t_r^i)|^2 \\ \mathcal{L}_b(\mathbf{w}, \boldsymbol{\lambda}_b) &= \frac{1}{2} \sum_{i=1}^{N_b} m(\lambda_b^i) |\mathcal{B}_{\mathbf{x},t}[u(\mathbf{x}_r^i, t_r^i; \mathbf{w})] - g(\mathbf{x}_b^i, t_b^i)|^2 \\ \mathcal{L}_0(\mathbf{w}, \boldsymbol{\lambda}_0) &= \frac{1}{2} \sum_{i=1}^{N_0} m(\lambda_0^i) |u(\mathbf{x}_0^i, 0; \mathbf{w}) - h(\mathbf{x}_0^i)|^2.\end{aligned}$$

- λ : pesos no negativos, auto-adaptativos entrenables (1 por cada punto de colocación!).
- “ $m(\lambda)$ ”: máscara definida en $[0, +\infty)$, no negativa, diferenciable, y estrictamente creciente.

Estrategia de entrenamiento auto-adaptativa (SA-PINN)

Entrenamiento min-max:

$$\min_w \max_{r, \lambda_b, \lambda_0} \mathcal{L}(w, \lambda_r, \lambda_b, \lambda_0).$$

$$w^{k-1} = w^k - \eta_k \nabla_w \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k)$$

GDesc

$$\lambda_r^{k+1} = \lambda_r^k + \eta_k \nabla_{\lambda_r} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k)$$

$$\lambda_b^{k+1} = \lambda_b^k + \eta_k \nabla_{\lambda_b} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k)$$

GAsc

$$\lambda_0^{k+1} = \lambda_0^k + \eta_k \nabla_{\lambda_0} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k)$$

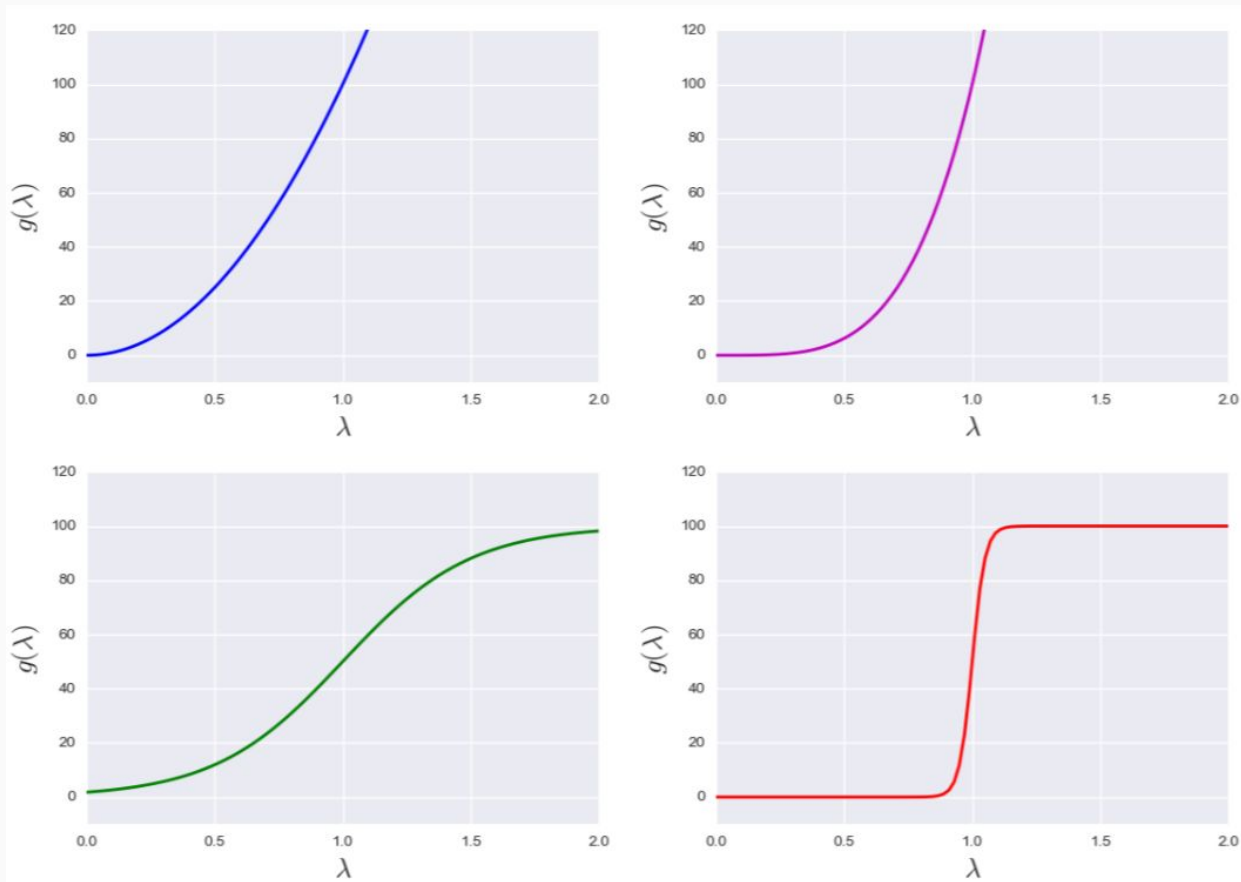
$$\nabla_{\lambda_r} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k) = \frac{1}{2} \begin{bmatrix} m'(\lambda_r^{k,1}) |\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x}_r^i, t_r^i; w^k)] - f(\mathbf{x}_r^1, t_r^1)|^2 \\ \dots \\ m'(\lambda_r^{k,N_r}) |\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x}_r^i, t_r^i; w^k)] - f(\mathbf{x}_r^{N_r}, t_r^{N_r})|^2 \end{bmatrix},$$

$$\nabla_{\lambda_b} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k) = \frac{1}{2} \begin{bmatrix} m'(\lambda_b^{k,1}) |\mathcal{B}_{\mathbf{x},t}[u(\mathbf{x}_b^i, t_b^i; w^k)] - g(\mathbf{x}_b^1, t_b^1)|^2 \\ \dots \\ m'(\lambda_b^{k,N_b}) |\mathcal{B}_{\mathbf{x},t}[u(\mathbf{x}_b^i, t_b^i; w^k)] - g(\mathbf{x}_b^{N_b}, t_b^{N_b})|^2 \end{bmatrix},$$

$$\nabla_{\lambda_0} \mathcal{L}(w^k, \lambda_r^k, \lambda_b^k, \lambda_0^k) = \frac{1}{2} \begin{bmatrix} m'(\lambda_0^{k,1}) |u(\mathbf{x}_0^1, 0; w^k) - h(\mathbf{x}_0^1, t_0^1)|^2 \\ \dots \\ m'(\lambda_0^{k,N_0}) |u(\mathbf{x}_0^i, 0; w^k) - h(\mathbf{x}_0^{N_0})|^2 \end{bmatrix}.$$

Gradientes >0 y
crecientes si
 $\lambda \neq 0$

SA-PINN: mascaras $m(\lambda)$



SA-PINN: Detalles de implementación

- SA-PINN con estrategia de entrenamiento Adam + L-BFGS: los pesos se entrenan durante la etapa Adam, y luego se dejan fijos en la etapa L-BFGS.
- SA-PINN original se implementa para la estrategia de entrenamiento full-batch, ya que los pesos entrenables están asociados a puntos de colocación específicos.
- Para utilizar minibatch o SGD, se utiliza una estrategia de regresión temporal-espacial para predecir los valores de los pesos en nuevos puntos muestreados.

SA-PINN: Implementación ecuación de Burgers viscosa

$$\begin{aligned} u_t + uu_x - (0.01/\pi)u_{xx} &= 0, \quad x \in [-1, 1], \quad t \in [0, 1], \\ u(0, x) &= -\sin(\pi x), \\ u(t, -1) &= u(t, 1) = 0. \end{aligned}$$

En Colab:

- Implementación vanilla PINN
- Implementación SA-PINN

Estrategias de sampleo de puntos de colocación

Sampleo no adaptativo uniformemente distribuido

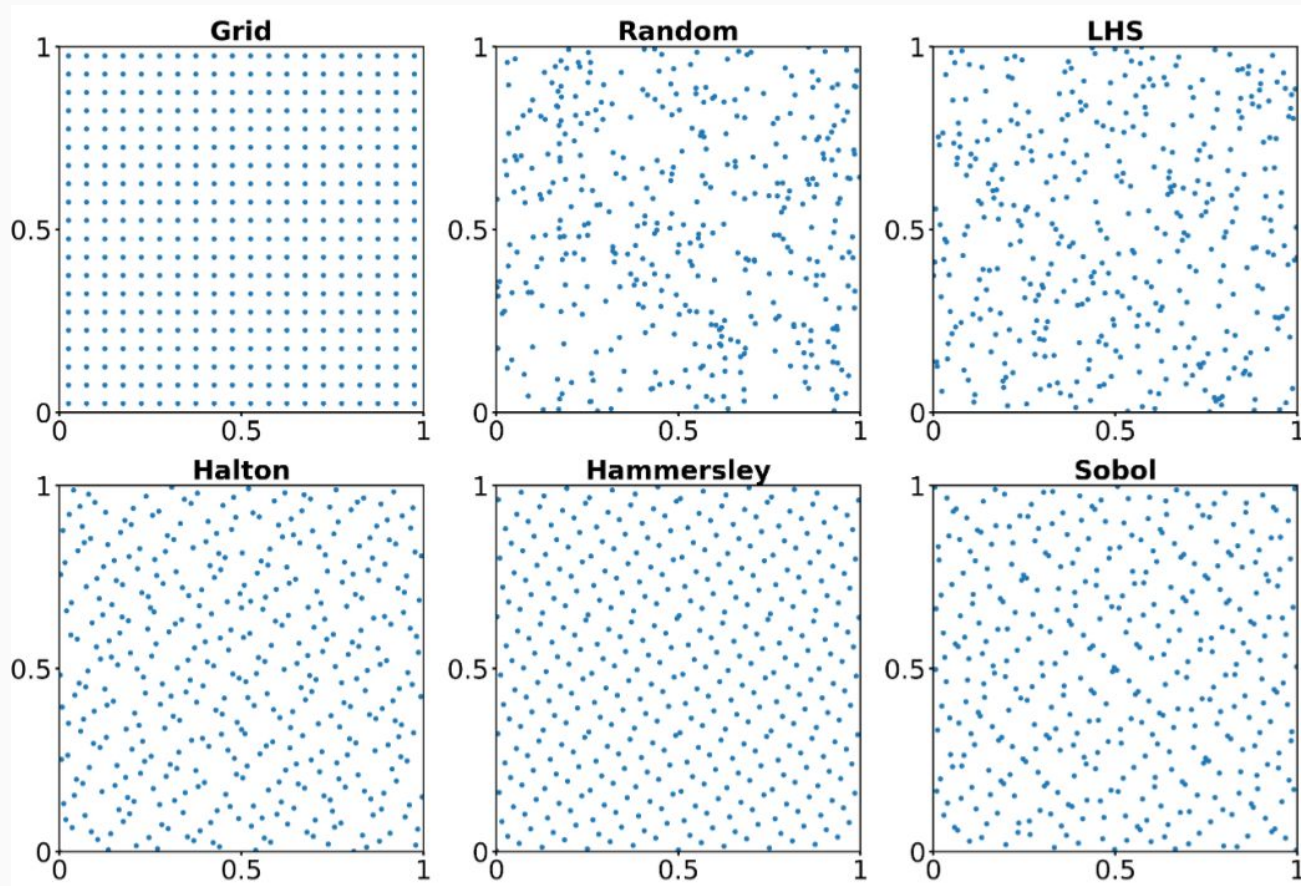
- Puntos de colocación fijos
 - Grilla uniforme equiespaciada (**Grid**)
 - Sampleo aleatorio uniforme (**Random**)
 - Sampleo por hipercubo latino (**LHS**)
 - Secuencias quasi-random de baja discrepancia:
 - De Halton (**Halton**)
 - De Hammersley (**Hammersley**)
 - De Sobol (**Sobol**)
- Puntos fijos con resampleo (**X-R**)

Sampleo adaptativo no uniforme

- Refinamiento adaptativo ávido basado en residuos (**RAR-G**)
- Distribución adaptativa basado en residuos (**RAD**)
- Refinamiento adaptativo basado en residuos con distribución (**RAR-D**)

Estrategias de sampleo de puntos de colocación

Sampleos fijos
generados en el
dominio $(0,1) \times (0,1)$
(Halton,
Hammersley y
Sobol: estrategias
pseudo-random)



Estrategias de sampleo de puntos de colocación

Estrategia de puntos fijos con resampleo X-R:

- Consiste en volver a generar el conjunto de puntos de colocación cada N epochs dentro del loop de entrenamiento
- Se puede utilizar cualquiera de las estrategias mencionadas, es decir $X=\{\text{Grid, Random, etc}\}$
- Por ejemplo, **Random-R** es útil cuando no es posible realizar un muestreo adaptativo

Estrategias de sampleo de puntos de colocación

Algoritmo estrategia RAR-G

1. Crear un conjunto inicial de puntos T mediante una estrategia de sampleo de puntos fijos
2. Entrenar una cierta cantidad de epochs
3. **Repetir:**
 - 3.1. Samplear un nuevo conjunto de puntos S_0 mediante una de las estrategias de punto fijo
 - 3.2. Computar los residuos de la PDE en el conjunto S_0
 - 3.3. Seleccionar el subconjunto de “m” puntos de S_0 que presenten los residuos más elevados (conjunto S)
 - 3.4. Unir los conjuntos: $T \leftarrow T \cup S$
 - 3.5. Entrenar una cierta cantidad de epochs
4. **Hasta:** alcanzar el número total de iteraciones, o se alcance el límite del número de puntos de colocación, etc.

Estrategias de sampleo de puntos de colocación

Construimos una PDF proporcional a la distribución de residuos (k y c son hiperparámetros):

$$p(\mathbf{x}) \propto \frac{\varepsilon^k(\mathbf{x})}{\mathbb{E}[\varepsilon^k(\mathbf{x})]} + c,$$

Algoritmo estrategia RAD

1. Crear un conjunto inicial de puntos T mediante una estrategia de sampleo de puntos fijos
2. Entrenar una cierta cantidad de epochs
3. **Repetir:**
 - 3.1. Crear un nuevo conjunto T a partir de un sampleo aleatorio de la PDF (valores por defecto: $k=1$, $c=1$)
 - 3.2. Entrenar una cierta cantidad de epochs
4. **Hasta:** alcanzar el número total de iteraciones, o se alcance el límite del número de puntos de colocación, etc.

Estrategias de sampleo de puntos de colocación

$$p(\mathbf{x}) \propto \frac{\varepsilon^k(\mathbf{x})}{\mathbb{E}[\varepsilon^k(\mathbf{x})]} + c,$$

Algoritmo estrategia RAR-D

1. Crear un conjunto inicial de puntos T mediante una estrategia de sampleo de puntos fijos
2. Entrenar una cierta cantidad de epochs
3. **Repetir:**
 - 3.1. Samplear “m” puntos de acuerdo a la PDF (conjunto S). Valores por defecto: $k=2$, $c=0$
 - 3.2. Unir los conjuntos: $T \leftarrow T \cup S$
 - 3.3. Entrenar una cierta cantidad de epochs
4. **Hasta:** alcanzar el número total de iteraciones, o se alcance el límite del número de puntos de colocación, etc.

Comparación de estrategias (Wu et al.)

Comparación norma L2 del error relativo de PINN vs solución exacta:

	Diffusion	Burgers'	Allen-Cahn	Wave
No. of residual points	30	2000	1000	2000
Grid	$0.66 \pm 0.06\%$	$13.7 \pm 2.37\%$	$93.4 \pm 6.98\%$	$81.3 \pm 13.7\%$
Random	$0.74 \pm 0.17\%$	$13.3 \pm 8.35\%$	$22.2 \pm 16.9\%$	$68.4 \pm 20.1\%$
LHS	$0.48 \pm 0.24\%$	$13.5 \pm 9.05\%$	$26.6 \pm 15.8\%$	$75.9 \pm 33.1\%$
Halton	$0.24 \pm 0.17\%$	$4.51 \pm 3.93\%$	$0.29 \pm 0.14\%$	$60.2 \pm 10.0\%$
Hammersley	$0.17 \pm 0.07\%$	$3.02 \pm 2.98\%$	$0.14 \pm 0.14\%$	$58.9 \pm 8.52\%$
Sobol	$0.19 \pm 0.07\%$	$3.38 \pm 3.21\%$	$0.35 \pm 0.24\%$	$57.5 \pm 14.7\%$
Random-R	$0.12 \pm 0.06\%$	$1.69 \pm 1.67\%$	$0.55 \pm 0.34\%$	$0.72 \pm 0.90\%$
RAR-G [3]	$0.20 \pm 0.07\%$	$0.12 \pm 0.04\%$	$0.53 \pm 0.19\%$	$0.81 \pm 0.11\%$
RAD	$0.11 \pm 0.07\%$	$0.02 \pm 0.00\%$	$0.08 \pm 0.06\%$	$0.09 \pm 0.04\%$
RAR-D	$0.14 \pm 0.11\%$	$0.03 \pm 0.01\%$	$0.09 \pm 0.03\%$	$0.29 \pm 0.04\%$

Comparación de estrategias (Wu et al.) - Casos seleccionados

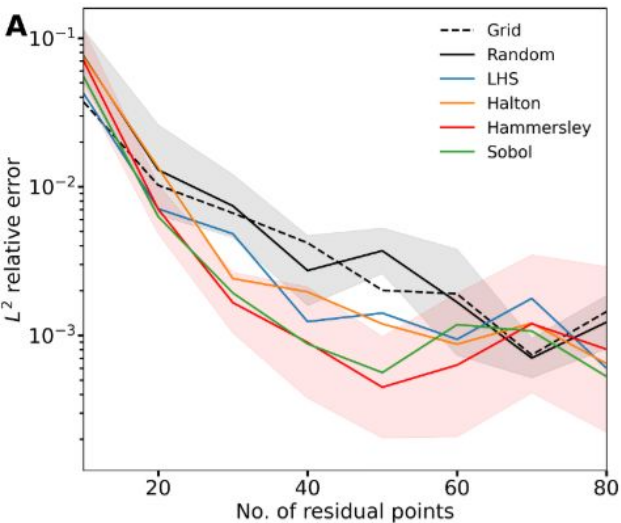
Problema de difusión 1D: $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + e^{-t} (-\sin(\pi x) + \pi^2 \sin(\pi x))$, $x \in [-1, 1], t \in [0, 1]$,

$$u(x, 0) = \sin(\pi x),$$

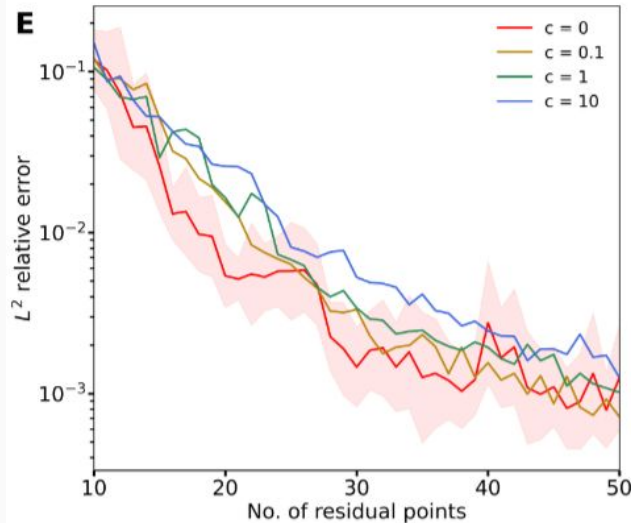
$$u(-1, t) = u(1, t) = 0,$$

Solución exacta: $u(x, t) = \sin(\pi x)e^{-t}$.

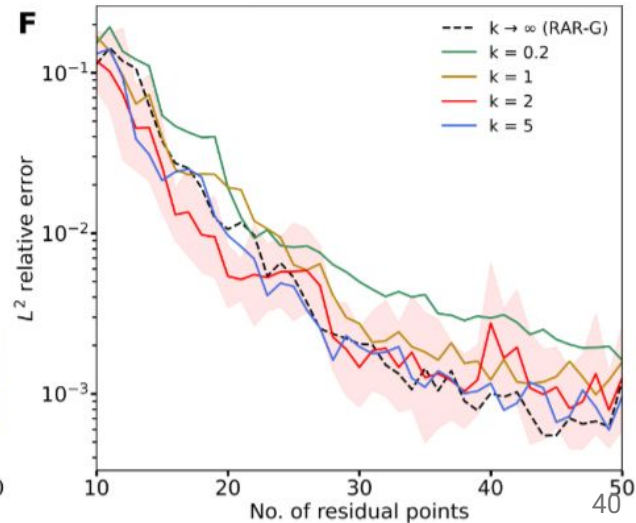
Muestreos de puntos fijos



RAR-D ($k=2$)



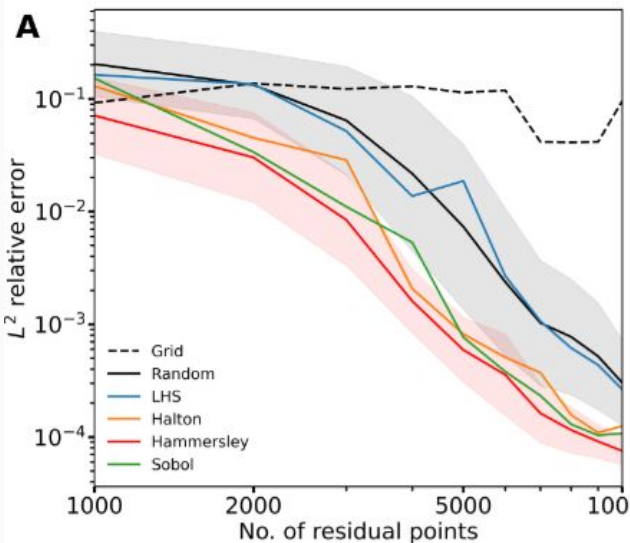
RAR-D ($c=0$)



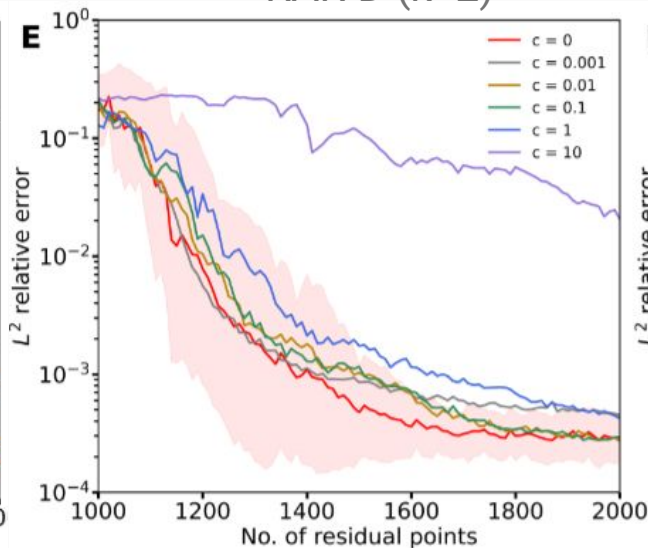
Comparación de estrategias (Wu et al.) - Casos seleccionados

Problema de Burgers 1D:
$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2}, \quad x \in [-1, 1], t \in [0, 1],$$
$$u(x, 0) = -\sin(\pi x),$$
$$u(-1, t) = u(1, t) = 0,$$

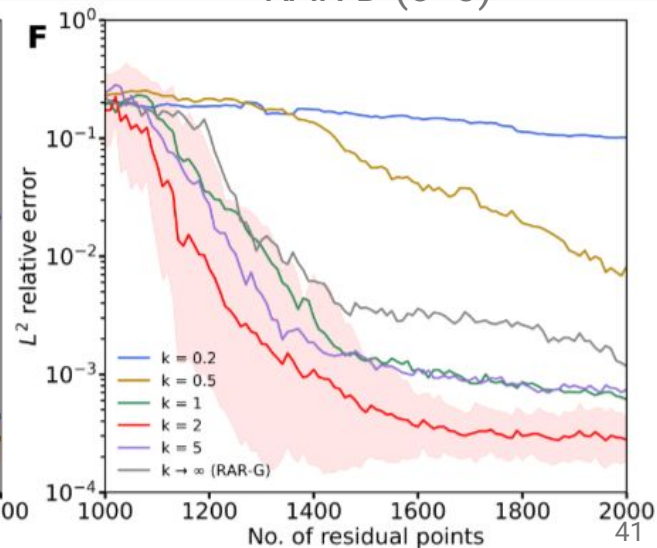
Muestreos de puntos fijos



RAR-D ($k=2$)



RAR-D ($c=0$)



Implementación en Colab

- Problema de difusión
- Problema de Burgers

Gradient-enhanced PINN (gPINN)

- Estrategia propuesta por Jeremy Yu (estudiante de 17 años de la escuela secundaria St. Mark School of Texas...)
- Consiste en aumentar la función de pérdida con información del gradiente del residuo
- Cuando se satisface la ecuación de gobierno, tanto el residuo como su gradiente se anulan:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_d} \right) = \mathbf{0}, \quad \mathbf{x} \in \Omega.$$

Función de pérdida
aumentada:

$$\mathcal{L} = w_f \mathcal{L}_f + w_b \mathcal{L}_b + w_i \mathcal{L}_i + \sum_{i=1}^{\infty} w_{g_i} \mathcal{L}_{g_i}(\boldsymbol{\theta}; \mathcal{T}_{g_i})$$

$$\mathcal{L}_{g_i}(\boldsymbol{\theta}; \mathcal{T}_{g_i}) = \frac{1}{|\mathcal{T}_{g_i}|} \sum_{\mathbf{x} \in \mathcal{T}_{g_i}} \left| \frac{\partial f}{\partial x_i} \right|^2$$

Gradient-enhanced PINN (gPINN)

Ejemplo: términos adicionales para la ecuación $\Delta u = f$

$$\text{En 1D: } \mathcal{L}_g = w_g \frac{1}{|\mathcal{T}_g|} \sum_{\mathbf{x} \in \mathcal{T}_g} \left| \frac{d^3 u}{dx^3} - \frac{df}{dx} \right|^2$$

$$\text{En 2D: } \mathcal{L}_{g_1} = w_{g_1} \frac{1}{|\mathcal{T}_{g_1}|} \sum_{\mathbf{x} \in \mathcal{T}_{g_1}} \left| \frac{\partial^3 u}{\partial x^3} + \frac{\partial^3 u}{\partial x \partial y^2} - \frac{\partial f}{\partial x} \right|^2,$$

$$\mathcal{L}_{g_2} = w_{g_2} \frac{1}{|\mathcal{T}_{g_2}|} \sum_{\mathbf{x} \in \mathcal{T}_{g_2}} \left| \frac{\partial^3 u}{\partial x^2 \partial y} + \frac{\partial^3 u}{\partial y^3} - \frac{\partial f}{\partial y} \right|^2.$$

Gradient-enhanced PINN (gPINN)

Ejemplo - ec. del calor:

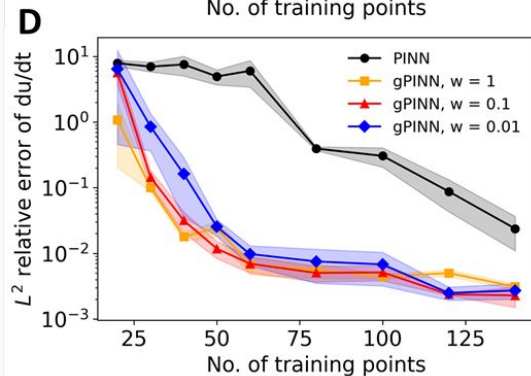
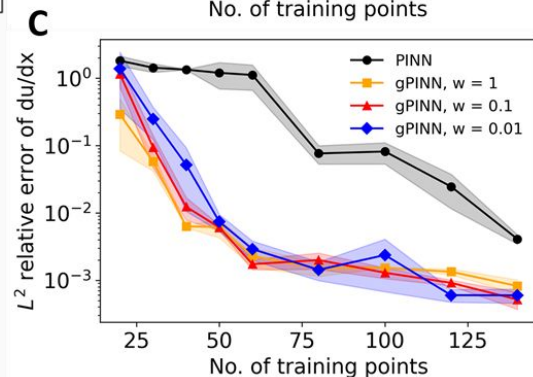
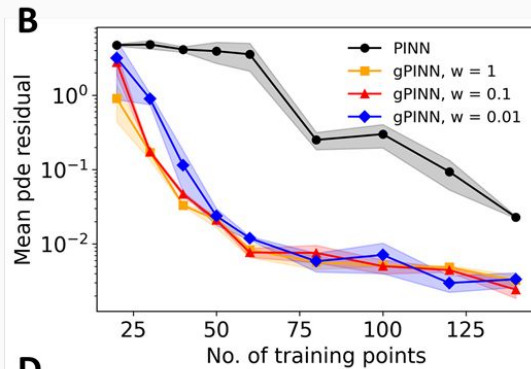
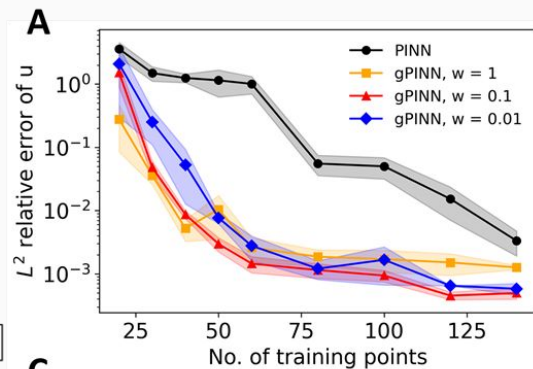
$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} + R(x, t), \quad x \in [-\pi, \pi], \quad t \in [0, 1],$$

$$R(x, t) = e^{-t} \left[\frac{3}{2} \sin(2x) + \frac{8}{3} \sin(3x) + \frac{15}{4} \sin(4x) + \frac{63}{8} \sin(8x) \right]$$

$$u(x, 0) = \sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8},$$

$$u(-\pi, t) = u(\pi, t) = 0,$$

$$\text{analytic solution for } u(x, t) = e^{-t} \left[\sum_{i=1}^4 \frac{\sin(ix)}{i} + \frac{\sin(8x)}{8} \right].$$



¿Dudas o
preguntas?